

Sky Team Brettspiel-Projekt Dokumentation

Inhaltsverzeichnis

Überblick.....	1
Projektkomponenten.....	1
Kernspiel-Logik.....	1
Grafische Benutzeroberfläche (GUI)	2
Tests	2
Versionskontrolle	2
Werkzeuge und Technologien	3
Projektstruktur im Detail.....	3
Entwicklungsrichtlinien	4
Design der Kernspiel-Logik	5
GUI-Design	8
Abhängigkeiten	8
Probleme	8
Meilensteine	8
Bereitstellung	9
Referenzen	9
Kontakt.....	9

Überblick

Das Sky Team Brettspiel-Projekt ist ein umfassendes Projekt mit dem Ziel, eine digitale Version des Brettspiels "Sky Team" zu entwickeln. Diese Version wurde in Java (libGDX Framework) programmiert. Ziel war es, sowohl die Kernspiel-Logik als auch eine visuell ansprechende grafische Benutzeroberfläche (GUI) zu erstellen. Das Projekt umfasste außerdem die Verwaltung von Assets, die Durchführung von Tests und den Einsatz eines Versionskontrollsystems, GitLab.

Obwohl das Projekt nicht vollständig abgeschlossen wurde, wurden wichtige Meilensteine erreicht.

Projektkomponenten

Kernspiel-Logik

- Die Spielmechanik wurde vollständig in einer Model-View-Controller (MVC)-Architektur implementiert. Diese Struktur trennt die Logik (Model), Darstellung (View) und Steuerung (Controller), um die Wartbarkeit und Skalierbarkeit zu verbessern.

Sky Team Brettspiel-Projekt Dokumentation

- Regeln und Mechaniken des Sky Team Brettspiels wurden implementiert.
- Die Klasse Game in der Komponente model ist die zentrale Klasse des Moduls und verwaltet die grundlegende Spiellogik.
- Die Board-Klasse enthält alle notwendigen Attribute und stellt sicher, dass die Interaktionen mit dem Spielbrett korrekt umgesetzt werden.
- Die Koordinaten für verschiedene Aufgaben auf dem Spielbrett wurden basierend auf Messungen der Textur des Brettspiels ermittelt.
- JSON-Datenstrukturen (Spielpläne), die für das Spiel verwendet wurden, wurden mithilfe der Jackson Library importiert.

Grafische Benutzeroberfläche (GUI)

- Eine interaktive Benutzeroberfläche wurde mit dem libGDX-Framework erstellt.
- Implementierte Bildschirme:
 - Main Screen: Das Hauptmenü des Spiels
 - Game Screen: Der Spielbildschirm, der das Spielbrett und die implementierten Spielfunktionen zeigt.
- Ein großes Problem war die Asset-Verwaltung: libGDX erfordert, dass Ressourcen nur zur Laufzeit aus einem speziellen "assets"-Ordner geladen werden. Dies führte zu Herausforderungen beim Testen und bei der Entwicklung.

Tests

- Unit-Tests wurden mit JUnit erstellt, um die Kernspiel-Logik zu validieren. Anfangs funktionierte JUnit jedoch nicht, da Gradle die Bibliothek nicht richtig erkannte. Dieses Problem wurde behoben, indem die Abhängigkeiten manuell angepasst wurden.
- Mock-Objekte wurden mit Mockito generiert, was sich jedoch als zeitaufwändig und komplex herausstellte. Diese Tests erforderten detaillierte Vorbereitungen und Anpassungen, insbesondere für komplexe Interaktionen.

Versionskontrolle

- GitLab wurde für die Versionskontrolle verwendet. Da das Projekt von mir alleine entwickelt wurde, waren keine komplexen Workflows oder Konfliktlösungen erforderlich.
- Änderungen wurden regelmäßig mit sinnvollen Commit-Beschreibungen dokumentiert, um den Fortschritt zu verfolgen. Dazu habe ich mit einer To-Do-Liste gearbeitet.

Sky Team Brettspiel-Projekt Dokumentation

Werkzeuge und Technologien

- Programmiersprache: Java
- Framework: libGDX
- Versionskontrolle: GitLab
- IDE: IntelliJ IDEA
- Testwerkzeuge: JUnit, Mockito
- JSON-Bibliothek: Jackson

Projektstruktur im Detail

Das Projekt besteht aus drei Hauptbereichen: den Ressourcen ("assets"), der Kernlogik ("core") und dem Desktop-Launcher ("lwjgl3").

- 1. Das Modul "assets":** Dieses Modul enthält alle benötigten Ressourcen wie Grafiken, JSON-Dateien und UI-Konfigurationen. Es ist das zentrale Verzeichnis für alles, was während der Laufzeit geladen werden muss. Hier befinden sich:
 - skins/: Enthält die grafischen Oberflächenelemente des Spiels.
 - ui/: Beinhaltet die Standard UI Texturen
 - "config.json" und "plans.json", die für die Konfiguration und Darstellung der Benutzeroberfläche benötigt werden.
- 2. Das Modul "core":** Hier befindet sich die gesamte Kernlogik des Spiels. Die Struktur ist klar in verschiedene Module unterteilt, um die Model-View-Controller (MVC)-Architektur zu unterstützen.
 - controller/: Enthält Klassen, die die Steuerung der Spielmechanik umsetzen, einschließlich der Verwaltung von Spieleraktionen.

Sky Team Brettspiel-Projekt Dokumentation

- model/: Hier liegen alle datenbezogenen Klassen wie das Spielfeld, Spieler und Spielzustände. Die zentrale Klasse ist Game, die die Kernlogik des Spiels steuert. Die Board-Klasse stellt die Daten und Attribute des Spielbretts bereit.
- view/: Zuständig für die Darstellung des Spiels, einschließlich der Haupt- und Spielbildschirme.
- test/: Dieser Bereich enthält die Unit-Tests für die Kernkomponenten

3. Das Modul "lwjgl3": Dieses Modul wiederum beherbergt den Desktop-Launcher, der das Spiel auf einer Desktop-Plattform ausführbar macht. Hier sind:

- java/: Beinhaltet die Klassen "Lwjgl3Launcher" und "StartupHelper", die für den Start des Spiels verantwortlich sind.
- resources/: Enthält verschiedene Icons für das Spiel, z. B. "libgdx16.png" bis "libgdx128.png", um unterschiedliche Auflösungen zu unterstützen.

4. Build-Skripte:

- build.gradle: Definiert die Abhängigkeiten und die Build-Konfiguration für das gesamte Projekt.
- settings.gradle: Organisiert die Modulstruktur des Projekts.

Diese klare Trennung der Projektbereiche erleichtert die Entwicklung, die Wartung und die Erweiterung des Spiels.

Entwicklungsrichtlinien

1. Clean Code

- Java Naming Conventions
- in Englisch geschrieben, um die Verständlichkeit zu verbessern
- Modularer Code wurde bevorzugt, um die Wiederverwendbarkeit zu gewährleisten.

2. Versionskontrollpraktiken

- GitLab wurde als Versionskontrollsystem verwendet
- Der Fokus lag auf einfachem und schnellem Management, da keine Zusammenarbeit im Team erforderlich war.

3. Tests

- Die Tests konzentrierten sich auf die Kernspiel-Logik.
- GUI-Komponenten wurden nicht mit JUnit/Mockito getestet, was zukünftige Tests und Verbesserungen erforderlich macht.

Sky Team Brettspiel-Projekt Dokumentation

- Integrationstests waren geplant, wurden jedoch nicht umgesetzt.

Design der Kernspiel-Logik

Game:

Rolle: Die zentrale Klasse des Spiels, die die gesamte Logik und den Zustand des Spiels verwaltet.

Funktionalitäten:

- Steuert den Spielablauf, einschließlich der Übergänge zwischen verschiedenen Spielphasen wie Setup, Spielrunde und Spielende.
- Koordiniert die Kommunikation zwischen den anderen Klassen wie Board, Player, und GameState.

Wichtige Attribute:

- Verweis auf die Klassen Board, Player, GameState und andere für die Spielmechanik relevante Objekte.

Player:

Rolle: Repräsentiert die Spieler des Spiels, einschließlich der spezifischen Rollen wie Pilot und Copilot.

Funktionalitäten:

- Speichert die Spielerdaten, einschließlich aktueller Ressourcen, Rollen und durchgeführter Aktionen.
- Unterstützt die Unterscheidung zwischen unterschiedlichen Rollen durch die Klasse Role.

Wichtige Attribute:

- Spielerrolle (Role), Punktestand, und Fähigkeiten.

Role:

Rolle: Definiert die spezifischen Eigenschaften und Fähigkeiten eines Spielers.

- **Funktionalitäten:**
- Legt fest, welche Aktionen ein Spieler in seiner Rolle durchführen kann.
- Unterstützt die Trennung von Verantwortlichkeiten zwischen Pilot und Copilot.

Board:

- **Rolle:** Modelliert das Spielbrett und alle darauf stattfindenden Aktionen.
- **Funktionalitäten:**
- Verwaltet die Anordnung und den Zustand von Elementen auf dem Spielbrett.
- Bietet Methoden, um die Interaktion mit Aufgaben oder Zonen auf dem Brett zu ermöglichen.

Sky Team Brettspiel-Projekt Dokumentation

- **Wichtige Attribute:**
- Struktur des Spielbretts (z. B. Zonen, Aufgabenkoordinaten).
- Dynamische Aktualisierungen basierend auf Spielaktionen.

GameState:

- **Rolle:** Hält den aktuellen Zustand des Spiels fest und stellt sicher, dass die Spielregeln eingehalten werden.
- **Funktionalitäten:**
- Überwacht die aktuelle Phase des Spiels und überprüft, ob Aktionen regelkonform sind.
- Speichert Informationen wie den aktiven Spieler, den aktuellen Punktestand und verbleibende Aufgaben.
- **Wichtige Attribute:**
- Aktueller Spielstatus, Turn-Informationen und Logs für Regelüberprüfungen.

Altitude:

- **Rolle:** Modelliert die Flughöhe des Flugzeugs, ein zentraler Mechanismus des Spiels.
- **Funktionalitäten:**
- Überwacht und aktualisiert die Flughöhe basierend auf Spieleraktionen.
- Stellt sicher, dass Höhenänderungen innerhalb der zulässigen Grenzen liegen.
- **Wichtige Attribute:**
- Aktuelle Höhe, Zielhöhe und zulässige Höhenbereiche.

FlightPlan:

- **Rolle:** Enthält die geplanten Aufgaben und Ziele für das Flugzeug während des Spiels.
- **Funktionalitäten:**
- Bietet den Spielern Anweisungen und spezifische Ziele, die sie erfüllen müssen.
- Verknüpft Aufgaben aus dem tasks-Paket mit bestimmten Flugbedingungen.
- **Wichtige Attribute:**
- Liste von Aufgaben, Flugrouten und zeitliche Beschränkungen.

Dice:

- **Rolle:** Repräsentiert Würfel, die für zufallsbasierte Mechaniken im Spiel genutzt werden.
- **Funktionalitäten:**
- Generiert zufällige Werte, die zur Bestimmung bestimmter Spielausgänge verwendet werden.
- Unterstützt mehrere Würfeltypen und -farben.
- **Wichtige Attribute:**
- Anzahl der Würfel, aktuelle Ergebnisse und Regeln für das Würfeln.

Plan:

- **Rolle:** Speichert und verwaltet kurzfristige Pläne und Strategien der Spieler.
- **Funktionalitäten:**

Sky Team Brettspiel-Projekt Dokumentation

- Hilft den Spielern, ihre Ressourcen und Aktionen effektiv zu koordinieren.
- **Wichtige Attribute:**
- Aufgabenprioritäten und verbleibende Aktionen.

Tasks

Das tasks-Paket enthält alle Klassen, die verschiedene Aufgaben repräsentieren, die Spieler während des Spiels erfüllen müssen. Jede Aufgabe ist auf ein spezifisches Spielziel ausgerichtet und unterscheidet sich durch Mechanik und Anforderungen.

Task (Abstrakte Klasse):

- **Rolle:** Die Basisklasse für alle Aufgaben.
- **Funktionalitäten:**
- Stellt gemeinsame Attribute und Methoden für alle spezifischen Aufgaben bereit.
- **Wichtige Attribute:**
- Aufgabenname, Status (erledigt/nicht erledigt) und Schwierigkeitsgrad.

Spezifische Aufgabenklassen:

- **AxisTask:**
 - Überprüft die Steuerung der Flugzeugachsen und stellt sicher, dass sie stabil bleiben.
- **BrakesTask:**
 - Erfordert die korrekte Bedienung der Bremsen, insbesondere bei der Landung.
- **CoffeeTask:**
 - Eine Aufgabe, die den Spielern ein Kaffee Token gibt.
- **FlapsTask:**
 - Überwacht die Einstellung der Landeklappen, die die Geschwindigkeitsgrenzen bewegt.
- **LandingGearTask:**
 - Überprüft, ob das Fahrwerk korrekt ein- oder ausgefahren wurde, bewegt die Geschwindigkeitsgrenzen.
- **RadioTask:**
 - Entfernt ein Flugzeug auf dem Plan.
- **SpeedTask:**

Sky Team Brettspiel-Projekt Dokumentation

- Überprüft, ob die Geschwindigkeit des Flugzeugs in den vorgegebenen Grenzen liegt und bewegt das Flugzeug.

GUI-Design

- ScreenManager: Ermöglicht die Verwaltung verschiedener Bildschirme.
- InputHandler: Verarbeitung von Benutzereingaben wie Klicks und Tastaturaktionen.
- Renderer: Verwendet libGDX, um Spielfiguren und Animationen darzustellen.

Abhängigkeiten

- libGDX: Bietet die grafische Grundlage des Spiels.
- JUnit: Wird für Unit-Tests verwendet.
- Mockito: Unterstützt beim Erstellen von Tests durch Mocking.
- Jackson: Verarbeitung von JSON-Daten.

Probleme

Während der Entwicklung des Projekts traten zunächst Probleme mit der Konfiguration von Gradle und den Abhängigkeiten auf. Insbesondere erkannte das System die JUnit-Bibliothek nicht korrekt, wodurch die Implementierung und Durchführung von Unit-Tests erheblich verzögert wurden. Dieses Problem wurde schließlich durch manuelle Anpassungen in den Build-Skripten behoben. Im weiteren Verlauf der Entwicklung stellte sich das Laden von Assets als Herausforderung dar, da libGDX erfordert, dass alle Ressourcen zur Laufzeit aus einem speziellen „assets“-Ordner geladen werden. Dies führte zu mehreren Komplikationen beim Testen und bei der Verwaltung der Assets. Ein weiteres Problem ergab sich bei der Bindung von Event-Listenern an die Drop Targets im Game Screen. Die korrekte Zuordnung und Reaktion auf Benutzereingaben war unerwartet komplex und erforderte umfangreiche Debugging- und Anpassungsarbeiten. Trotz dieser Hürden konnten diese Herausforderungen durch gezielte Fehlersuche und die Anpassung der Implementierung überwunden werden.

Meilensteine

1. Implementierung der Kernspiel-Logik

- Design und Entwicklung der Kernklassen für die Spielmechanik.
- Durchführung detaillierter Tests mit JUnit und Mockito.

Sky Team Brettspiel-Projekt Dokumentation

2. GUI-Integration

- Entwicklung eines funktionalen Hauptmenüs und Spielscreens.
- Integration von Benutzerinteraktionen mit Spielmechaniken.

3. Neue Technologien und Herausforderungen

- Grundlagen der Versionskontrolle mit Git erlernt und erfolgreich angewendet.
- Nutzung von Mockito für Mock-Objekte und dessen Herausforderungen gemeistert.
- Arbeit mit Unit-Tests vertieft
- Zum ersten Mal eine grafische Benutzeroberfläche (GUI) mit libGDX programmiert und dabei grundlegende Kenntnisse in der GUI-Entwicklung erworben.

Bereitstellung

- Erstellung eines funktionalen Builds mit Gradle.
- Lwjgl3Launcher

Referenzen

- <https://libgdx.com/dev/>
- https://www.brettspielversand.de/mediafiles/spieleanleitungen/kosmos/03-0576_SkyTeam_Anleitung.pdf
- <https://docs.gitlab.com/>

Kontakt

- Projektinhaber: Emrah Zehic Inf 3
- E-Mail: emrah.zehic@hof-university.de